Sign Language Sentence Recognition

System Architecture

MediaPipe + OpenCV + Conv1D/BiLSTM

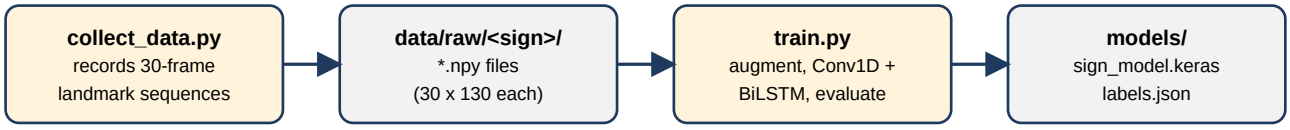
Item	Value
Purpose	Recognize signs from a webcam in real time and display the resulting sentence on the video frame
Recognition type	Fixed-vocabulary sign classification from 30-frame sequences, assembled into sentences
Language / stack	Python 3.10+, OpenCV, MediaPipe Tasks (HandLandmarker), TensorFlow/Keras, scikit-learn
Entry points	collect_data.py (record), train.py (train), realtime.py (run)
Status	Prototype (version 0.1.0)

1. Design goals

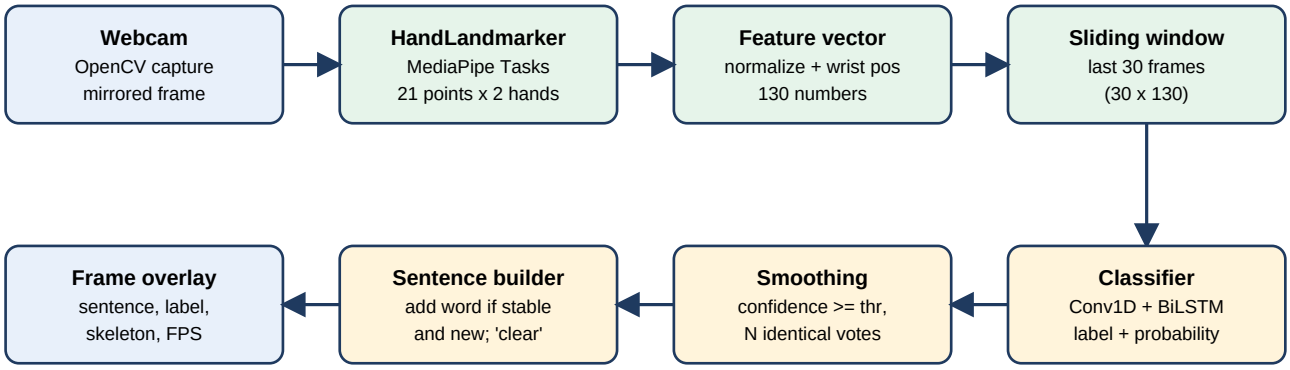
- **Robust to background, lighting and skin tone:** the model sees hand landmarks (numbers), not pixels.
- **Light and fast:** 130 features per frame; a small model that runs in real time on a laptop CPU.
- **Simple to extend:** the vocabulary, thresholds and paths all live in one file (config.py).
- **Consistent features:** the same extraction code is used for recording, training and live inference.
- **Honest scope:** isolated signs assembled into a sentence, not continuous signing.

2. System overview

A. Offline pipeline (done once per vocabulary)



B. Real-time pipeline (realtime.py, runs for every camera frame)



Shared by both pipelines: utils.py (HandLandmarker + feature extraction) and config.py (all settings).

The offline pipeline turns your recordings into a trained model. The real-time pipeline reuses the same feature extraction, feeds a rolling 30-frame window to the model, and converts stable predictions into words.

3. Components

File	Responsibility	Key functions
config.py	Single source of truth: vocabulary (ACTIONS), SEQ_LEN, feature sizes, paths, training and inference thresholds	constants only
utils.py	MediaPipe wrapper, feature extraction, drawing helpers. Downloads hand_landmarker.task on first run	create_hands, detect, hands_present, extract_features, draw_hands, put_text_box
collect_data.py	Records (30 x 130) arrays per sample to data/raw/<sign>/<n>.npy with a countdown before each recording	main, next_index

File	Responsibility	Key functions
train.py	Loads data, stratified 80/20 split, augments train split only, trains, prints report and confusion matrix, saves model + labels.json	load_dataset, augment, build_model, main
realtime.py	Live loop: buffer, predict, smooth, build sentence, draw overlay; keyboard controls and optional speech	main, speak

4. Data flow and tensor shapes

Stage	Shape	Notes
Camera frame	(480, 640, 3)	BGR, mirrored with cv2.flip
HandLandmarker output	0 to 2 hands x 21 x (x, y, z)	Normalized image coordinates; includes Left/Right label
Per-hand features	65	21 x 3 wrist-relative, scaled by wrist-to-middle-finger distance (63) + wrist (x, y) (2)
Frame feature vector	130	Left slot + right slot; missing hand = zeros
Sequence (model input)	(30, 130)	30 frames, about 1 second at 30 FPS
Model output	(num_classes,)	Softmax probabilities, one per entry in ACTIONS

Why these features? Subtracting the wrist and scaling by hand size makes the hand *shape* independent of where the hand is and how far it is from the camera. The absolute wrist position is added back because in sign language the location of a sign carries meaning. Two hands are placed in fixed slots (sorted by horizontal position when both are visible) so the model sees a consistent layout.

5. Model architecture

Layer	Output shape	Purpose
Input	(30, 130)	One sequence of frame features
Conv1D(64, k=3, ReLU) + BatchNorm	(30, 64)	Local short-term motion patterns, stable training
BiLSTM(64, return_sequences) + Dropout(0.3)	(30, 128)	Temporal context, reading forward and backward
BiLSTM(64)	(128)	Summarize the whole gesture
Dense(64, ReLU) + Dropout(0.3)	(64)	Classification features, regularization
Dense(N, softmax)	(N)	Probability for each sign

Training setup: Adam (learning rate 1e-3), sparse categorical cross-entropy, up to 80 epochs, EarlyStopping (patience 12, restores best weights), ReduceLROnPlateau, batch size 32, stratified 80/20 split. Augmentation (noise, small time shift, small scale) is applied to the training split only so the test score stays honest.

6. Real-time decision logic

Raw model output flickers, so a prediction becomes a word only after several checks:

- **Idle detection:** if no hand is seen for more than MAX_NO_HAND frames, the state is forced to *idle*.
- **Prediction rate:** the model runs every PREDICT_EVERY frames once the 30-frame buffer is full.
- **Confidence gate:** predictions below CONF_THRESHOLD (0.85) are treated as *uncertain*.
- **Stability window:** the last STABLE_N (6) predictions must all be identical.
- **Novelty rule:** the word must differ from the last committed word. Returning to *idle* resets this, so the same word can be signed twice.
- **Special signs:** *idle* never produces text; *clear* erases the sentence.

Keyboard controls: q quit, c clear sentence, b delete last word, s speak the sentence (if pyttsx3 is installed).

7. Repository layout

```
sign-sentence/
|-- config.py              vocabulary, paths, thresholds
|-- utils.py               MediaPipe wrapper, features, drawing
|-- collect_data.py        step 1: record data
|-- train.py               step 2: train model
|-- realtime.py            step 3: live recognition
|-- pyproject.toml         packaging, ruff/pytest configuration
|-- requirements.txt       simple dependency list
|-- README.md              usage guide
|-- AGENTS.md              rules for AI agents / contributors
|-- development_plan.md    roadmap and checklist
|-- architecture.pdf       this document
|-- data/raw/<sign>/*.npy  recorded sequences (git-ignored)
`-- models/                sign_model.keras, labels.json,
                           hand_landmarker.task (git-ignored)
```

8. Configuration reference

Setting	Default	Effect
ACTIONS	15 entries	Sign vocabulary; must include idle
SEQ_LEN	30	Frames per sample (about 1 s)
NUM_SEQUENCES	40	Samples recorded per sign
AUGMENT_COPIES	3	Extra augmented copies of each training sample
CONF_THRESHOLD	0.85	Minimum probability to accept a prediction
STABLE_N	6	Identical consecutive predictions needed to commit a word
MAX_NO_HAND	10	Frames without hands before state becomes idle
PREDICT_EVERY	2	Run the model every N frames (speed vs responsiveness)
CAMERA_INDEX	0	Which camera to open

9. Design decisions and trade-offs

Decision	Reason	Trade-off
Landmarks instead of raw pixels	Robust to background/lighting; tiny input; fast	Loses face and body detail unless added
MediaPipe Tasks API	Legacy mp.solutions is removed in newer releases	Needs a downloaded .task model file
Sequence model (BiLSTM)	Signs involve motion over time	Fixed 30-frame window; cannot segment continuous signing
Hysteresis smoothing	Prevents flicker and accidental words	Adds a short delay before a word appears
Flat module layout	Simple to understand and run	Not a package; use pip install -e . for entry points

10. Known limitations and extension points

- **Hands only:** face and body pose carry grammar in real sign languages. Add MediaPipe pose/face landmarkers and extend extract_features (and retrain).
- **Isolated signs:** continuous recognition needs CTC or Transformer models and datasets such as WLASL, MS-ASL or How2Sign.
- **Gloss output:** the sentence is a sequence of signs, not grammatical text; a rule-based or language-model step can rewrite it.
- **Generalization:** a model trained on one person will struggle with others. Record multiple people and split by person.
- **Deployment:** export to TFLite/ONNX for lower latency; wrap in FastAPI + WebRTC for a web UI.
- **Feature changes:** any change to feature size/order invalidates recordings and models; regenerate both.

Responsible use: this is a prototype and not a certified communication aid. Validate with Deaf and hard-of-hearing signers before any real-world use.